

UNIVERSITÉ PARIS NANTERRE

MASTER 1 MIAGE

Optimisation Combinatoire

GraphBench

Réfutation automatique de conjectures en théorie des graphes

Art Hashani & Ryad Messaoudi

Année universitaire 2025–2026

13 Mai 2026

Dépôt GitHub : https://github.com/ryad1602/projet_graphes

Conjectures réfutées	100 / 100
Score solver seul	183,89 s
Score avec FunSearch	140,53 s
Gain de performance	−23,6 %

Stack technique : Python 3, NetworkX, PuLP/CBC, GROQ API (Llama-3.3-70b)

Table des matières

1	Introduction et objectif	3
1.1	Forme des conjectures	3
1.2	Validité d'un contre-exemple	3
1.3	Score de violation et métrique d'évaluation	3
2	Heuristique de recherche	4
2.1	Architecture de recherche en trois phases	4
2.2	Phase 0 : Balayage de l'atlas	4
2.3	Phase 1 : Population initiale ciblée	4
2.4	Phase 2 — Recuit simulé (SA)	5
2.5	Calcul exact des invariants NP-difficiles	5
3	Architecture FunSearch	6
3.1	Représentation des solutions	6
3.2	Boucle d'optimisation	6
3.3	Rôle du LLM : Générateur de diversité motivée	6
3.4	Résultats de l'apprentissage	7
4	Résultats expérimentaux	7
4.1	Performance globale et impact de FunSearch	7
4.2	Répartition par classe structurelle	7
4.3	Distribution temporelle de la recherche	8
4.4	Analyse des cas critiques	8
4.5	Propriétés des contre-exemples générés	8
5	Discussion scientifique	8
5.1	Typologie des conjectures réfutées	8
5.2	Invariants et complexité de calcul	9
5.3	Analyse de l'efficacité des mutations	9
5.4	Impact de FunSearch sur l'heuristique	9
5.5	Apport du LLM et perspectives	10

1. Introduction et objectif

Ce projet s’inscrit dans le cadre du TD noté de Master 1 MIAGE. L’objectif est de **réfuter automatiquement 100 conjectures mathématiques** issues de la théorie des graphes. Pour chaque conjecture, il s’agit de construire un contre-exemple : un graphe appartenant à une classe imposée et violant strictement une inégalité entre deux invariants.

1.1 Forme des conjectures

Chaque conjecture est définie par une relation de la forme :

$$Y \leq f(X) \quad \text{ou} \quad Y \geq f(X) \quad (1)$$

Où X et Y sont des invariants de graphes et f un polynôme spécifié dans le benchmark. Le périmètre d’étude couvre 25 invariants classés en deux catégories :

- **Invariants polynomiaux** : Ordre (n), taille (m), degrés (min, max, moyen), densité, diamètre, rayon, triangles, matching, indices de Randić, Zagreb, harmonique, proximité, remoteness, et valeurs propres.
- **Invariants NP-difficiles** : Domination (γ), domination totale (γ_t), domination indépendante (i), indépendance (α), couverture par sommets (β) et clique (ω).

1.2 Validité d’un contre-exemple

Pour qu’un graphe G soit considéré comme un contre-exemple valide, il doit impérativement satisfaire quatre conditions :

1. **Classe structurelle** : Le graphe doit respecter la contrainte imposée (connexe, arbre, ou sans-griffe connexe).
2. **Calcul exact** : Les invariants X et Y doivent être calculés avec précision.
3. **Violation stricte** : L’inégalité de la conjecture doit être strictement contredite.
4. **Formatage** : Le graphe doit être exporté au format `graph6`.

1.3 Score de violation et métrique d’évaluation

Afin d’unifier le traitement algorithmique, nous utilisons un *score de violation* δ , défini de sorte qu’il soit **positif** dès qu’un contre-exemple est trouvé :

$$\delta(G) = Y(G) - f(X(G)) \quad \text{si la conjecture est } Y \leq f(X)$$

$$\delta(G) = f(X(G)) - Y(G) \quad \text{si la conjecture est } Y \geq f(X)$$

Dans les deux cas : G est un contre-exemple $\iff \delta(G) > 0$.

Cette définition symétrique permet à nos heuristiques de toujours maximiser la même grandeur, indépendamment du sens de l'inégalité initiale.

Le succès du benchmark est mesuré par la somme des temps de résolution, incluant une pénalité de 120 s par échec :

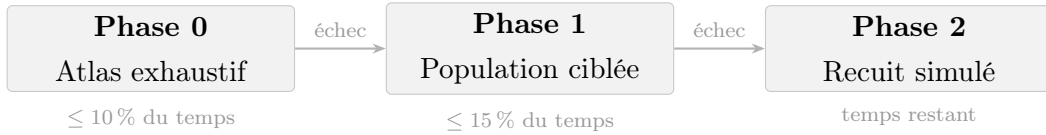
$$\text{Score Total} = \sum_{\text{réfutées}} t_i + (120 \times \text{Nombre d'échecs}) \quad (2)$$

L'objectif de performance est donc de minimiser ce score global.

2. Heuristique de recherche

L'heuristique, implémentée dans `solver.py`, articule trois phases complémentaires. Elle s'exécute avec un temps limite de 60 s par conjecture, s'interrompant immédiatement dès qu'un contre-exemple est validé.

2.1 Architecture de recherche en trois phases



2.2 Phase 0 : Balayage de l'atlas

Tous les graphes connexes jusqu'à $n = 7$ sommets issus de l'atlas NetworkX (≈ 830 graphes) sont testés. Cette phase est déterministe et quasi-instantanée.

Efficacité : 68 conjectures sur 100 sont réfutées durant cette phase en moins d'une seconde.

2.3 Phase 1 : Population initiale ciblée

Si l'atlas ne suffit pas, nous générons des familles de graphes extrémaux basées sur les invariants impliqués dans la conjecture :

Famille	Propriété recherchée	Invariants cibles
Étoiles $K_{1,n}$	Degré max élevé, faible λ_2	Δ, λ_2
Chemins P_n	Diamètre et remoteness maximaux	diam, rem
Cycles C_n	Régularité degré / diamètre	δ, Δ
Bipartis complets $K_{p,q}$	Matching et indépendance extrêmes	μ, α
Double-étoiles S_{k_1,k_2}	Domination indépendante vs couverture	i, β
Barbells K_k -arête- K_k	Faible connectivité algébrique	λ_2, Δ
Line graphs $L(H)$	Structure spécifique (sans-griffe)	tous
Réguliers aléatoires	Régularité contrôlée	Zagreb, Randić

2.4 Phase 2 — Recuit simulé (SA)

Le recuit simulé explore l'espace des graphes par mutations successives pour maximiser le score de violation δ .

Mutations et réparation. L'algorithme adapte ses mutations à la classe du graphe : **12 mutations** pour les graphes généraux, **6 mutations** pour les graphes sans-griffe et **4 mutations** pour les arbres. Après chaque mutation, une procédure **repair** garantit la validité structurelle (reconnexion des composantes, retrait de griffes induites, arbre couvrant minimal pour les arbres).

Mécanismes anti-stagnation.

- **Réchauffement** : Si aucune amélioration n'est observée après 50 mutations, la température est triplée pour sortir d'un optimum local.
- **Hard Reset** : Après 35 % du temps sans progrès, la population est intégralement régénérée.

2.5 Calcul exact des invariants NP-difficiles

Pour les invariants complexes (ex : nombre de domination γ), nous utilisons la programmation linéaire entière (ILP) via PuLP/CBC :

$$\gamma(G) = \min \sum_{v \in V} x_v \quad \text{s.c.} \quad x_v + \sum_{u \in N(v)} x_u \geq 1 \quad \forall v \in V, \quad x_v \in \{0, 1\} \quad (3)$$

Pour optimiser les performances, le solveur utilise une approche à deux niveaux :

- **Calcul rapide** : Approximations gloutonnes $O(n^2)$ durant les mutations du SA.
- **Calcul exact** : L'ILP n'est déclenché que si le score approximé suggère un potentiel contre-exemple.

Cette stratégie réduit d'un facteur ≈ 100 les appels ILP sans compromettre la correction : tout contre-exemple retourné est garanti par un calcul exact.

3. Architecture FunSearch

Le module FunSearch (`funsearch.py`) optimise automatiquement les **poids de sélection des mutations** du recuit simulé.

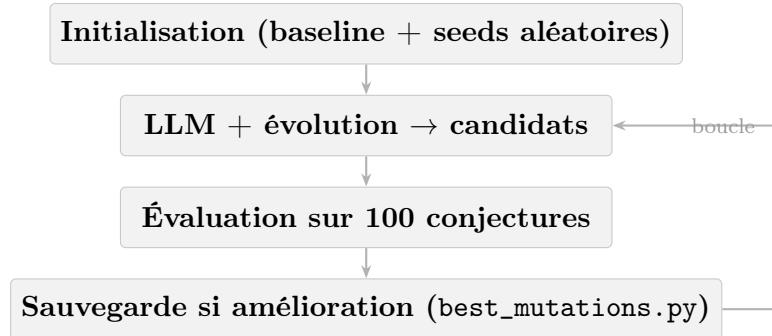
3.1 Représentation des solutions

Les solutions sont modélisées par des vecteurs de **22 réels**, représentant l'importance relative de chaque mutation :

$$\mathbf{w} = \left(\underbrace{w_1, \dots, w_{12}}_{\text{Général}}, \underbrace{w_{13}, \dots, w_{16}}_{\text{Arbre}}, \underbrace{w_{17}, \dots, w_{22}}_{\text{Sans-griffe}} \right) \in [0, 05; 5, 0]^{22} \quad (4)$$

Chaque vecteur est évalué sur l'intégralité du benchmark. Le meilleur candidat est sauvegardé de manière persistante dans `best_mutations.py`.

3.2 Boucle d'optimisation



Sources de candidats. Deux moteurs génèrent de nouveaux vecteurs :

- **LLM (Llama-3.3-70b)** : Reçoit les poids actuels avec leurs étiquettes sémantiques et les statistiques de réussite. Il propose des ajustements motivés par le raisonnement structurel.
- **Évolution** : Mutations gaussiennes (fines $\sigma = 0,08$ et larges $\sigma = 0,5$), crossovers du *top-4* et vecteurs aléatoires.

3.3 Rôle du LLM : Générateur de diversité motivée

Le LLM agit comme un expert capable d'anticiper l'effet d'une mutation sur les invariants. Voici un extrait représentatif des raisonnements produits (consignés dans `funsearch_history.json`) :

« Privilégier `m_twins` pour les graphes sans-griffe car ils créent des voisinages similaires favorables à la domination, et `m_subdivide` pour les arbres pour impacter directement le diamètre. »

Cette approche permet d’explorer des zones de l’espace de recherche inaccessibles à une simple mutation aléatoire.

3.4 Résultats de l’apprentissage

L’extrait suivant présente la configuration retenue, obtenue lors d’une itération aboutissant à un taux de réussite de 100 % sur le benchmark :

Listing 1 – Extrait de results/best_mutations.py

```

1 MUTATION_WEIGHTS = {
2     'general': [0.74, 1.83, 1.82, 0.19, 0.78, 1.29, 0.70,
3               1.60, 1.44, 0.98, 1.76, 1.21],
4     'tree': [0.50, 1.08, 1.43, 0.84],
5     'claw_free': [1.38, 0.99, 1.06, 0.77, 1.16, 1.22]
6 }
```

Tendances observées : `rm_edge` (1,83) et `add_node` (1,82) sont privilégiés pour les graphes généraux, tandis que `subdivide` (1,43) est jugé crucial pour les arbres. Cette séparation rend FunSearch optionnel : le solver reste autonome et ne charge ces poids que s’ils sont disponibles.

4. Résultats expérimentaux

4.1 Performance globale et impact de FunSearch

L’évaluation comparative montre une amélioration significative des performances temporelles grâce à l’optimisation des poids de mutation par FunSearch.

Configuration	Conjectures réfutées	Score (Temps total)
Solver seul (poids uniformes)	100 / 100	183,89 s
Solver + poids optimisés FunSearch	100 / 100	140,53 s
Gain de performance	—	−43,36 s (−23,6 %)

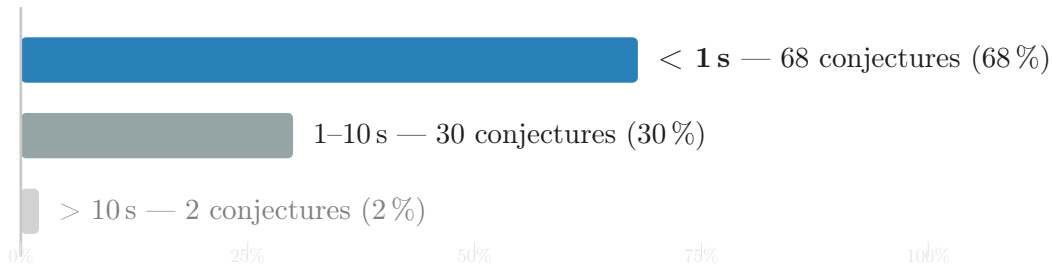
4.2 Répartition par classe structurelle

Le tableau suivant détaille l’efficacité du solver selon la complexité structurelle des graphes demandés. Les arbres s’avèrent les plus simples à réfuter, tandis que les graphes sans-griffe présentent la plus grande variabilité de temps.

Classe	Réfutées	Temps moyen	Temps max
Connexes généraux (44)	44 / 44	2,14 s	6,59 s
Sans-griffe (50)	50 / 50	1,77 s	43,54 s
Arbres (6)	6 / 6	0,23 s	0,82 s
Total (100)	100 / 100	1,84 s	43,54 s

4.3 Distribution temporelle de la recherche

La majorité des conjectures sont réfutées de manière quasi-instantanée, validant l'efficacité de la Phase 0 (Atlas) et de la Phase 1 (Population ciblée).



4.4 Analyse des cas critiques

Certaines conjectures exigent une exploration profonde du recuit simulé, notamment celles impliquant la *total domination* sur des graphes sans-griffe.

ID	Invariants (X vs Y)	Classe	Temps
6903	matching vs total_domination	Sans-griffe	43,54 s
6574	total_domination vs independence	Sans-griffe	22,65 s
2051	indep_domination vs vertex_cover	Connexe	6,59 s
2252	randic_index vs total_domination	Connexe	6,41 s
1466	domination vs indep_domination	Connexe	6,22 s

4.5 Propriétés des contre-exemples générés

L'heuristique ne se contente pas de trouver des violations marginales; elle identifie souvent des structures divergeant fortement des conjectures initiales.

Indicateur	Valeur
Violation médiane (δ)	0,55
Violation maximale	835,44
Taille moyenne des graphes	26,4 sommets
Étendue de l'ordre (n)	3 à 121 sommets

5. Discussion scientifique

5.1 Typologie des conjectures réfutées

Les 68 conjectures résolues en moins d'une seconde partagent deux caractéristiques : elles impliquent des invariants exclusivement polynomiaux et appartiennent à la classe des arbres ou des graphes connexes généraux.

Les **arbres** constituent la classe la plus aisée (0,23s en moyenne). L'absence de cycles contraint fortement l'espace de recherche, et les mutations spécifiques comme **leaf** et **subdivide** permettent de modifier directement les invariants structurels clés sans rompre la validité de la classe.

5.2 Invariants et complexité de calcul

Les huit conjectures dépassant 4 secondes impliquent toutes au moins un invariant NP-difficile. L'invariant **total_domination_number** (γ_t) apparaît dans six d'entre elles. La difficulté rencontrée est double :

1. **Coût algorithmique** : Le calcul ILP devient onéreux ($\approx 50\text{--}200$ ms pour $n \in [20, 35]$). De plus, l'approximation gloutonne (*greedy*) a tendance à sous-estimer γ_t , ce qui réduit la fiabilité du score rapide utilisé par le recuit simulé.
2. **Rareté structurelle** : L'espace des graphes où γ_t atteint des valeurs extrêmes est peu dense, rendant les contre-exemples plus difficiles à "capturer" par mutation aléatoire.

5.3 Analyse de l'efficacité des mutations

L'optimisation par FunSearch révèle des tendances cohérentes avec la théorie des graphes :

- **Graphes généraux** : **rm_edge** (1,83) et **add_node** (1,82) dominant. Supprimer une arête augmente naturellement l'indépendance (α) et la domination (γ), tandis que l'ajout de sommets amplifie les invariants proportionnels à l'ordre n .
- **Arbres** : **subdivide** (1,43) est privilégiée car elle allonge les chemins internes et modifie la structure de domination sans risquer de créer un cycle.
- **Sans-griffe** : **add_edge** (1,38) et **twins** (1,22) sont favorisées. La création de jumeaux produit des voisinages identiques, ce qui est une stratégie connue pour impacter les invariants de domination.

À l'inverse, **rm_node** (0,19) est systématiquement pénalisée, car la réduction de l'ordre diminue la plupart des invariants que l'heuristique cherche à maximiser pour violer les bornes.

5.4 Impact de FunSearch sur l'heuristique

Bilan du gain : L'apport de FunSearch est mesurable avec une réduction de **23,6 %** du temps total de calcul.

Le gain se concentre essentiellement sur les conjectures de difficulté intermédiaire (1 à 10s), là où le recuit simulé dispose d'assez de temps pour que le choix préférentiel des mutations influe sur la vitesse de convergence. L'effet est nul sur les cas triviaux (Phase 0 suffisante) et limité sur les cas très complexes où le goulot d'étranglement est le solveur ILP lui-même.

5.5 Apport du LLM et perspectives

Le LLM (Llama-3.3-70b) a agi comme un **accélérateur de diversité motivée**. Plutôt que de procéder par tâtonnements purement statistiques, il propose des sauts dans l'espace des poids guidés par un raisonnement sémantique (ex : corrélation entre `subdivide` et diamètre). Les journaux de bord (`funsearch_history.json`) confirment que l'IA identifie correctement les leviers structurels avant même que l'évolution n'en valide l'efficacité.

Limites et perspectives. Deux pistes d'amélioration sont envisageables :

- **Spécialisation des poids :** Définir des poids spécifiques à chaque conjecture (ou groupe d'invariants) plutôt que par classe de graphe.
- **Seuil ILP adaptatif :** Apprendre un seuil de déclenchement de l'ILP plus fin pour limiter les calculs exacts inutiles sur les mutations non prometteuses.